

Lab 6: OpenMP in Practice - Detailed Study Guide

This extensively expanded guide covers everything you need to know about OpenMP for C/C++ parallel programming. OpenMP uses pre-processor directives (`#pragma omp`) to automatically inject threading code, saving you from writing low-level `pthread` boilerplate.

1. Core Principles & Execution Model

Compilation

You absolutely **MUST** compile with the flag. Without it, the compiler treats all pragmas as comments, and the code runs entirely sequentially.

- **Compile command:** `gcc -fopenmp program.c -o program`

The Fork-Join Model

Unlike traditional programs where 1 thread runs from start to finish:

1. **Start:** The program begins with exactly 1 "Master" thread.
2. **Fork:** When the Master thread hits an `#pragma omp parallel` block, it "forks" (spawns/wakes up) a team of worker threads.
3. **Execute:** All threads execute the block inside the braces simultaneously.
4. **Join:** At the closing brace `}`, an implicit barrier exists. All threads wait there. Once everyone finishes, the worker threads sleep, and only the Master thread continues linearly.

Directives

- `omp_set_num_threads(N)`: Tell OpenMP to spawn exactly N threads at the next fork.
 - `omp_get_thread_num()`: Returns the calling thread's ID (0 is the Master).
-

2. Work Sharing Directives

You rarely want every thread doing the exact same work. You want to "share" a single large workload among them.

A. Parallel For

If you have a 10,000 iteration loop and 4 threads, OpenMP automatically assigns 2,500 iterations to each thread.

```
#pragma omp parallel for
for(int i = 0; i < 10000; i++) {
    process(data[i]);
}
```

B. Scheduling Techniques (`schedule(type, chunk_size)`)

When compiling `#pragma omp parallel for`, you can specify how iterations are divided.

1. **`schedule(static)`**: Chunk size is set at compile/start time. Thread 0 gets iterations 0-99. Thread 1 gets 100-199.
 - *Best use*: Loop operations that take the exact same time (like array additions). Lowest overhead.
2. **`schedule(dynamic)`**: Threads are given a small "chunk" of iterations (default 1). When they finish, they immediately grab the next available chunk.
 - *Best use*: Highly unpredictable loops (like raytracing, or prime number factoring where testing 999983 takes longer than testing 2). Prevents fast threads from sitting idle.
3. **`schedule(guided)`**: A smart dynamic schedule. OpenMP starts by giving out massive chunks of iterations, and as the loop nears completion, it gives out smaller and smaller chunks. Reduces the overhead of dynamic scheduling while keeping load balanced.

C. Parallel Sections

For executing completely distinct logic paths concurrently.

```
#pragma omp parallel sections
{
    #pragma omp section
    download_files(); // Thread 0 does this

    #pragma omp section
    play_audio_stream(); // Thread 1 does this
}
```

3. Dealing with Race Conditions

A Race Condition occurs when multiple threads write to the same regular variable simultaneously, causing data corruption.

1. `critical`

Allows only 1 thread inside the protected block at a time. It works universally but is incredibly slow because it basically un-parallelizes your code at that bottleneck.

```
#pragma omp critical
{
    total += cost; // Only 1 thread gets here at a time
}
```

2. `atomic`

A hardware-level instruction that guarantees an extremely fast lock, but ONLY applies to a single simple statement like `x++`, `x--`, `x += y`.

```
#pragma omp atomic
total += cost; // Super fast!
```

3. `reduction(operator:variable)` (The Champion!)

If you are doing aggregations (summing arrays, finding maximums, finding products), `reduction` is fundamentally better than locks.

- **How it works:** OpenMP creates a secret *local clone* of the variable for every single thread. Thread 1 updates its own local target seamlessly with zero locks. Thread 2 does the same. At the very end of the loop, OpenMP merges all the local variables into the original global variable automatically.

```
long sum = 0;
#pragma omp parallel for reduction(+:sum)
for(int i=0; i<N; i++) {
    sum += array[i]; // No locks needed inside loop, runs at maximum CPU speed
}
```

4. Exam Cheat Sheet: Adapting OpenMP Directives

If the question says...	What to inject into the code...	Code Snippet
"Parallelize this independent standard <i>for</i> loop"	Add a parallel for directive above the loop.	<code>#pragma omp parallel for</code>
"Loop iterations involve highly varying calculation times"	Change the scheduling strategy to dynamic to prevent idle threads.	<code>#pragma omp parallel for schedule(dynamic)</code>
"The loop updates a single shared sum or counter"	Add a reduction clause to prevent race conditions safely.	<code>#pragma omp parallel for reduction(+:overall_sum)</code>
"Ensure only one thread prints X at a time"	Wrap the <code>printf</code> or critical array update in a critical section.	<code>#pragma omp critical</code>
"Ensure all threads finish Phase 1 before any thread starts Phase 2"	Insert a manual barrier between the phases.	<code>#pragma omp barrier</code>

If the question says...	What to inject into the code...	Code Snippet
"Two completely different functions must run concurrently"	Use the <code>sections</code> directive to assign one function per thread.	<pre>#pragma omp parallel sections { #pragma omp section func1(); #pragma omp section func2(); }</pre>

5. Multi-Layer Exam Preparation: Solved Mock Questions

Question A: Parallel Loop Bugs

Problem Statement: A student tries to parallelize a loop computing a Fibonacci sequence: `fib[i] = fib[i-1] + fib[i-2]`. They add `#pragma omp parallel for`. The output becomes completely corrupted. Why did this happen, and can it be fixed in OpenMP?

Step-by-step Solution:

- 1. **Identify the bug:** Fibonacci sequences inherently have a **Loop-Carried Data Dependency**. Iteration `i=5` absolute MUST wait for iteration `i=4` and `i=3` to be fully computed.
- 2. **Explain the failure:** OpenMP assigns iteration `5` to Thread 2 and it might execute *before* Thread 1 even computes iteration `4`. Thread 2 reads uninitialized garbage memory.
- 3. **The Fix:** You cannot fix this with OpenMP loop directives. `parallel for` strictly mathematically requires iterations to be completely independent of each other. The student must use a sequential loop, or a fundamentally different algorithm (like matrix exponentiation) to parallelize Fibonacci.

Question B: Scheduling Decisions

Problem Statement: You have a loop evaluating an extremely complex integral. Array elements are checked: `if(x[i] < 0) { continue; } else { do_complex_math(); }`. Currently, `schedule(static)` is used, but CPU performance monitors show 3 cores are sitting at 0% usage while 1 core is at 100% usage for a long time.

- 1. What is this phenomenon called?
- 2. What is the single line code change required to fix it?

Step-by-step Solution:

- 1. **Phenomenon Identification:** This is called **Load Imbalance**. Because `static` assigns chunks blindly by index, it's possible Thread 0 got all the positive numbers (requiring heavy math) and threads 1, 2, and 3 got all the negative numbers (which `continue` instantly). Threads 1-3 finished in milliseconds and went idle waiting at the implicit barrier.
- 2. **The Fix:** Change the schedule to dynamically grant workload to whoever finishes early. `#pragma omp parallel for schedule(dynamic)`

Question C: Prime Code Review

Problem Statement: Look at the following code.

```
int count = 0;
#pragma omp parallel
{
    #pragma omp for
    for(int i=2; i<=20000; i++) {
        if(is_prime(i)) {
            count++;
        }
    }
}
printf("%d primes found\n", count);
```

1. Identify the race condition.
2. Provide two completely different OpenMP ways to fix it. State which one is superior for performance.

Step-by-step Solution:

1. **The Race Condition:** `count++` is inside a parallel region. Multiple threads will read, increment, and write back simultaneously, losing counts.
2. **Fix 1 (Atomic):** Place `#pragma omp atomic` immediately above `count++`; . This forces hardware synchronization.
3. **Fix 2 (Reduction - Superior):** Replace `#pragma omp for` with `#pragma omp for reduction(+:count)`.
4. **Why Reduction is Superior:** `atomic` still causes a minor traffic jam inside the loop if two threads find a prime at the same time. `reduction` completely eliminates traffic jams because everyone updates their own local `count`. It is much faster for a 20,000 iteration loop.